

Matrices

Programación I

UNRN

Universidad Nacional
de **Río Negro**

03

Septiembre

Programación I

Clase 06: Arreglos de N dimensiones

Universidad Nacional de Río Negro | Ingeniería en Sistemas | Ciclo 2026

Prof. Daniel Teira/Diego Diaz

Sede Bariloche

Contenidos de la clase

- Repaso de “arreglos”
- Matrices
- Matrices y funciones
- Operaciones con matrices
- Argumentos main
- Algunas librerías útiles

Ayuda: <https://www.jdoodle.com/c-online-compiler>

0 Repaso de arreglos

Definición: estructura de datos estática que permite almacenar múltiples valores del mismo tipo en posiciones de memoria contiguas bajo un mismo nombre

tipo *nombre*[TAMANO];

```
int arreglo[] = {0, 1, 2, 3, 4};
```



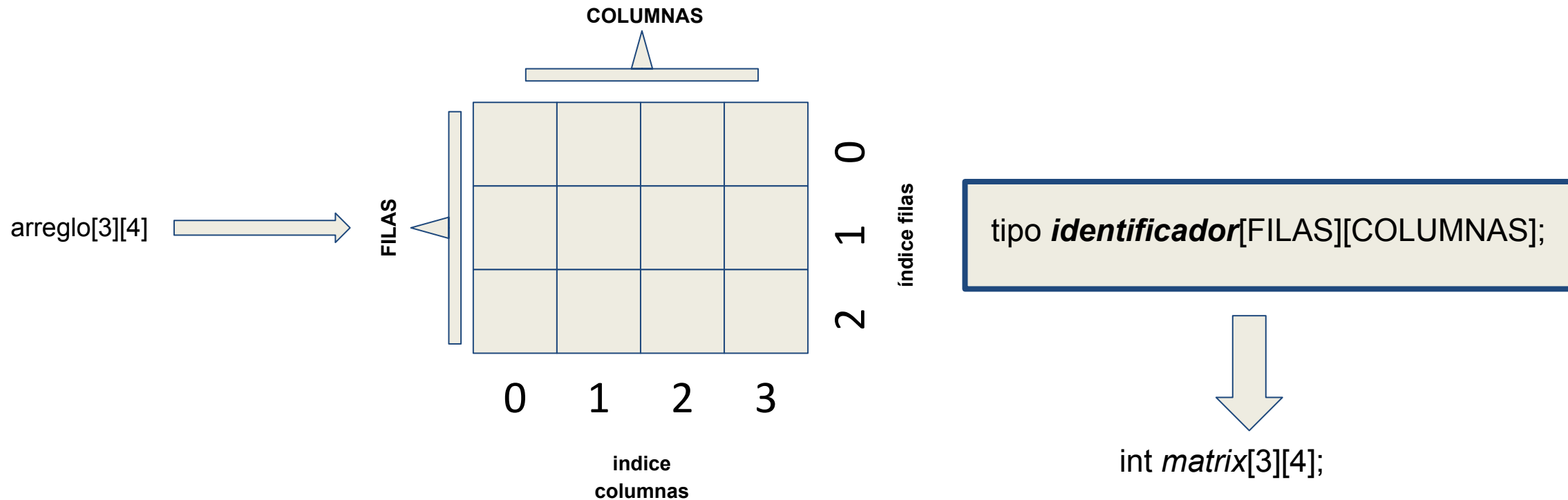
0	1	2	3	4
---	---	---	---	---



```
int arreglo[MAX];  
for (int i = 0; i < MAX; i++)  
{  
    arreglo[i] = i;  
}
```

1 Matrices

Definición: arreglo rectangular de números o datos ordenados en filas (líneas horizontales) y columnas (líneas verticales).
Recuerden, siempre el sufijo de FILAS VA PRIMERO, y el sufijo de COLUMNAS VA SEGUNDO



1 Matrices: Inicialización

```
tipo identificador[FILAS][COLUMNAS];  
int matriz[3][3];
```

Y en la memoria, la matriz es....



1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---

```
int matriz[3][3] = {{1, 2, 3},  
                    {4, 5, 6},  
                    {7, 8, 9}};
```

1	2	3
4	5	6
7	8	9

1 Matrices: Inicialización (ejercicio)

Funcionan igual que los arreglos

Carga

```
int numeros[3][3];
int suma = 0;
for (i = 0; i < 3; i++)
{
    for (j = 0; j < 3; j++)
    {
        numeros[i][j] = suma++;
    }
}
```

Impresión

```
for (i = 0; i < 3; i++)
{
    printf("matriz=\n");
    for (j = 0; j < 3; j++)
    {
        printf("[%d]",numeros[i][j]);
    }
    printf("\n");
}
```

Los arreglos *estáticos* sólo pueden ser creados de un tamaño fijo al compilar  Fallará

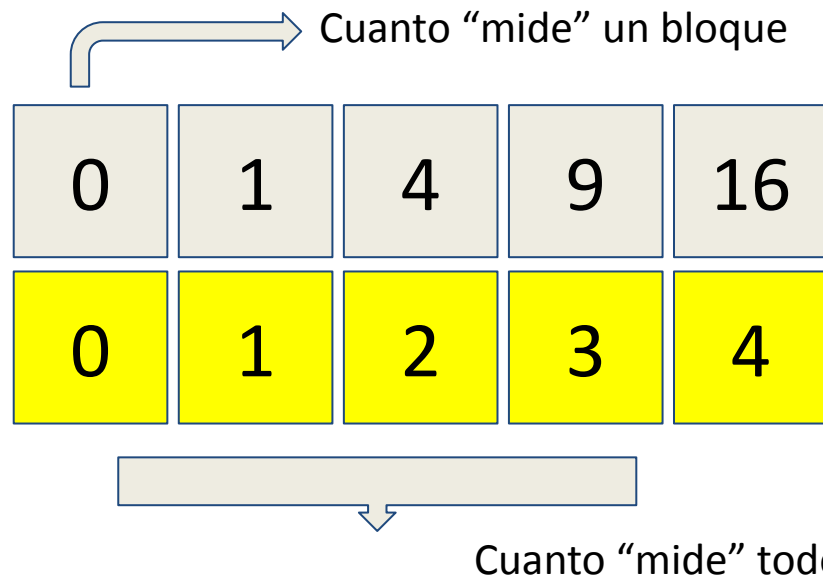
1 Matrices y funciones

Repasemos con un arreglo: Para este `int arreglo[]` de `size_t` largo

```
size_t largo(int arreglo[])  
{  
    return sizeof(arreglo) / sizeof(arreglo[0]);  
}
```



Cuando pasas un arreglo como argumento a una función, este se degrada a un puntero (process conocido como pointer decay). Por lo tanto, `sizeof(arreglo)` no devuelve el tamaño total del arreglo, sino el tamaño del puntero (usualmente 4 u 8 bytes).



NO es la manera de hacerlo!,
debo pasarle matriz (vector) y
tamaño

1 Matrices y funciones: tamaño de matriz

Ahora, si queremos “medir” el tamaño de una matriz, sucede lo mismo, También decae a dirección de memoria, entonces la solución es:

```
#include <stdio.h>
```

```
void procesar_matriz(size_t filas, size_t columnas, int matriz[][columnas])
{
    printf("La matriz tiene %zu filas y %zu columnas.\n", filas, columnas);
}
```

```
int main() {
    int mi_matriz[3][4];
```

ZU: codificador para tipo t_size y
sin signo

```
// En el mismo ámbito donde se creó, sizeof Sí da el tamaño total
```

```
printf("Bytes totales en main: %zu\n", sizeof(mi_matriz)); // 3 * 4 * 4 bytes = 48
```

```
    procesar_matriz(3, 4, mi_matriz);
    return 0;
```

```
}
```

1 Matrices y funciones: porque solo defino “Columns” a la función?

	0	1	2	3	4
0	A	B	C	D	E
1	F	G	H	I	J
2	K	L	M	N	O

```
void muestra(size_t filas,  
            size_t columnas,  
            int matrix[filas][columnas]);
```

```
char matrix[3][5];
```

En la memoria de la computadora, las matrices de C **no existen como bloques de dos dimensiones**. La memoria RAM es un renglón largo y continuo (una sola dimensión).

Cuando declaras `int mi_matriz[n][m]`, el compilador guarda los $n*m$ números en fila, uno detrás de otro:

	0	1	2	3	4		0	1	2	3	4		0	1	2	3	4
0	A	B	C	D	E	1	F	G	H	I	J	2	K	L	M	N	O

1 Operaciones con matrices: ejercicios

Ejercicio #1: Declarar dos matrices estáticas de 2×2 con números enteros a tu elección. Crea una tercera matriz vacía para guardar el resultado. Imprimir la matriz resultante en pantalla dándole forma de cuadrícula, aprovechando el salto de línea `\n` y el modificador de ancho (como `%4d`).

Ejercicio #2: Crea una matriz de 3×3 inicializada con valores secuenciales del 1 al 9. Pídele al usuario (mediante `scanf`) que ingrese un número entero multiplicador. Multiplica cada celda de la matriz por ese número y muestra el tablero actualizado.

Ejercicio #3: cree una calculadora que multiplique de matrices de $n \times m$.

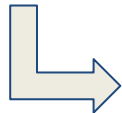
1 Argumentos main

Si queremos que nuestro programa reciba configuraciones desde la línea de comandos al momento de ejecutarse, debemos cambiar `int main()` por su versión completa formal

`int main(int argc, char *argv[])`



Recuerdan que se comentó que “main” tenía el aspecto de una función?



argv (Argument Vector - Arreglo de textos): Es un arreglo de cadenas de texto (`char *[]` o arreglo de strings). Aquí es donde se guardan, una por una, las palabras reales que el usuario escribió separadas por espacios.



argc (Argument Count - Contador): Es un número entero (`int`). Le dice a tu programa exactamente cuántas palabras se escribieron en la consola para ejecutarlo.

El programa **siempre se cuenta a sí mismo** como el primer argumento.

1 Argumentos main Ejemplo

<https://www.jdoodle.com/c-online-compiler>

Si usan gcc: gcc calculadora.c -o calculadora

```
#include <stdio.h>
#include <stdlib.h> // Libreria obligatoria para usar atoi()

int main(int argc, char *argv[]) {
    if (argc != 3) {
        printf("Error: Debes ingresar exactamente dos números.\n");
        printf("Uso correcto: calculadora <num1> <num2>\n");
        return 1; // Un return distinto de 0 indica que el programa falló
    } //Verificó

    int numero1 = atoi(argv[1]); //atoi convierte de cadena de texto a entero
    int numero2 = atoi(argv[2]); //atoi convierte de cadena de texto a entero

    // 3. Hacemos la matemática y mostramos el resultado
    int suma = numero1 + numero2;
    printf("La suma de %d y %d es: %d\n", numero1, numero2, suma);

    return 0;
}
```

```
./calculadora 15 25
```

1 Argumentos main Ejercicio

Ejercicio #3: Crea un programa que reciba el nombre y apellido del usuario desde la consola al momento de ejecutarse. Si el usuario olvida poner alguno de los dos datos (o pone de más), el programa debe indicar el error y mostrarle el "uso correcto". Si lo hace bien, debe imprimir un saludo personalizado.

(Salida: "Error: Debes ingresar exactamente tu nombre y apellido...")

Salida: "¡Hola Cosme Fulanito! Bienvenido al mundo de los argumentos en C.")

2 Librerías string.h & stdlib.h

atof - string a double

atoi - string a int Esta es particularmente importante, ya que cuando uno lee por consola (Scanf) usa chars, pero si quiero, por ejemplo, sumar 2 numeros, debo convertirlos a enterios

atol - string a long int

atoll - string a long long int

strtof - string a float

Otros métodos:

`int strcmp( char* izq, char* dch);` Compara lexicográficamente dos cadenas

`int strcmp(char* str1, const char* str2);`

Compara str1 con str2, indicando un ordenamiento basado en el código de los caracteres de ambas cadenas.

- <0 el primer carácter que no coincide es de valor menor
- 0 iguales
- >1 el primer carácter que no coincide es de mayor valor

`char* strcpy( char *dst, char *src);`

Copia la cadena src en dst, con el terminador en el largo de src.